



Факультет компьютерных наук
ОП "Современные компьютерные науки"

Москва, 2026

Методы поиска кандидатов на графических ускорителях в рекомендательных системах

Нигматуллин Роман Максимович

Научный руководитель: Хрыльченко Кирилл Ярославович

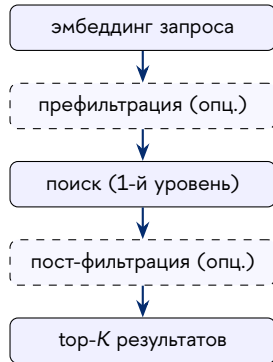


Двухстадийная архитектура рекомендательной системы: поиск кандидатов сужает каталог из 10^6 – 10^9 объектов до 10^2 – 10^3 для последующего ранжирования.

Жёсткие требования к этапу поиска:

- **Скорость** — от единиц до сотен мс на запрос
- **Полнота** — Recall@K определяет потолок качества всего конвейера
- **Фильтрация** по атрибутам объектов (жанр, язык, регион, лицензия)
- **Память** — каталоги до 10^9 , рост размерности эмбедингов

Тенденция: переход от статических ANN-индексов к поиску на основе модели.





Обзор: методы векторного поиска

- **Квантизация и инвертированные индексы (IVF):** Product / Scalar Quantization, ScaNN.
- **Графовые алгоритмы:** HNSW, DiskANN, NVIDIA CAGRA (GPU), SONG.
- **Векторные СУБД на GPU:** Milvus. **Недостатки:** низкоуровневый код (C++), вызывают разрывы вычислительного графа PyTorch, отсутствие гибкой атрибутной фильтрации прямо на GPU.

Поиск на основе модели (LiNR, LinkedIn 2024)

Поиск кандидатов выражен как единый слой `nn.Module` (индекс + вычисления). Оценки сходства (скоры) вычисляются на лету с одновременной фильтрацией внутри графа нейросети.

Проблема: проприетарный код, алгоритмы V1–V3 недоступны для открытого тестирования и использования, что и мотивирует создание нашего решения.



Цель работы и её вклад

Цель. Создать открытый фреймворк `torchretrieve` на PyTorch + Triton для поиска кандидатов на GPU с поддержкой атрибутной фильтрации, который легко встраивается в граф вычислений модели и совместим с `torch.compile`.

Решаемые проблемы и вклад:

- **Проблема закрытости кода:** Воспроизведены индустриальные алгоритмы LiNR V1–V3 под единым открытым API.
- **Проблема нехватки релевантных кандидатов (liquidity):** Реализована фильтрация прямо в процессе поиска, предотвращающая вытеснение релевантных объектов.
- **Высокое потребление памяти:** Предложен новый INT8-вариант **LiNR V4**, снижающий объем индекса в два раза при $\geq 95\%$ полноты.
- **Скорость работы:** Разработан авторский алгоритм **QuantizedIVF** (IVF + INT8 + фильтр Блума), работающий в одном Triton-ядре (ускорение до $\approx 9\times$).



Формальная постановка задачи

Пусть $X \in \mathbb{R}^{N \times D}$ — каталог из N объектов, $Q \in \mathbb{R}^{B \times D}$ — батч запросов. Атрибутный фильтр задает допустимые объекты с помощью бинарной маски $M \in \{0, 1\}^{B \times N}$.

Задача: для каждого запроса q_i найти:

$$l_i = \arg \text{top-}K \{ q_i^\top x_j \mid M[i, j] = 1 \}.$$

Ограничения системы:

- требования к времени ответа t_{med} ;
- пиковое потребление памяти GPU;
- интеграция в граф PyTorch без разрывов.

Методология оценки

Базовый метод: точный векторный поиск (KNN) полным перебором с маскированием.

Все приближённые алгоритмы сравниваются по скорости и полноте (Recall@K) относительно этого базового метода.



LiNR V1 и V2 — точный поиск с фильтрацией

V1 — постфильтрация

- Считаем все скоры сходства.
- Заменяем отфильтрованные на $-\infty$.

V2 — префильтрация

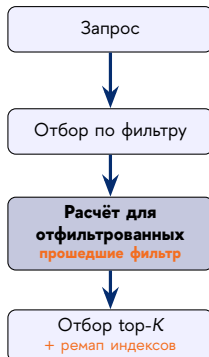
- Сначала отбираем прошедшие фильтр объекты.
- Считаем скоры сходства только для них.

Алгоритмы из статьи LiNR
(Borisyuk et al., LinkedIn, 2024).

V1 — постфильтр



V2 — префильтр



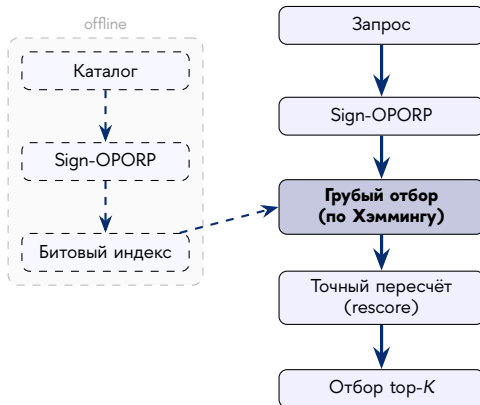


LiNR V3 — Sign-OPORP + расстояние Хэмминга

Двухэтапный поиск (Sign-OPORP, Li 2023)

- **Сжатие:** эмбединги бинаризуются до 1 бита на размерность.
- **Грубый отбор:** фильтрация + быстрый поиск по расстоянию Хэмминга.
- **Точный пересчёт:** точные скоры (rescore) для пула кандидатов.

$W = \lceil D/64 \rceil$; P_c — размер пула кандидатов для точного пересчёта.



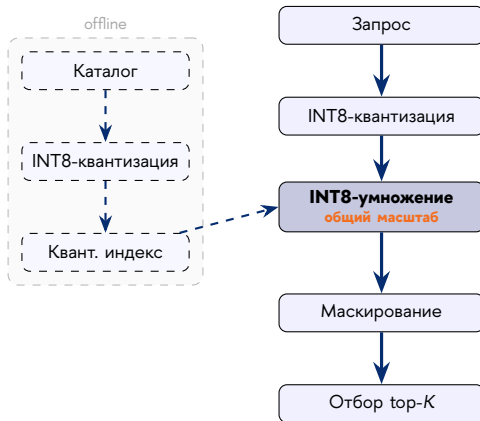


LiNR V4 — INT8-квантизация (вклад работы)

Модификация V1 с INT8-квантизацией

- Скалярная INT8-квантизация каталога с **общим масштабом** (scale).
- Поиск top- K осуществляется сразу в формате INT32.
- Деквантизация в числа с плавающей точкой только для итогового ответа.

Общий масштаб s_x позволяет находить top- K без деквантизации всех скоров.



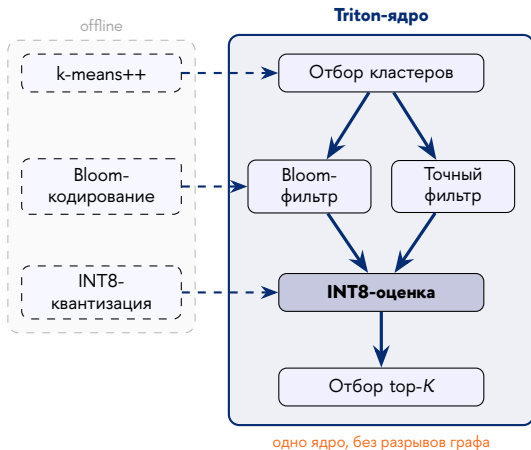


QuantizedIVF — авторский алгоритм с фильтрацией

Triton-ядро со встроенным фильтром

- **Кластеризация:** отбор ближайших кластеров к запросу.
- **Фильтрация:** внутри кластеров отсекаются нерелевантные объекты.
- **INT8-поиск:** для прошедших фильтрацию кандидатов скорости вычисляются в INT8.

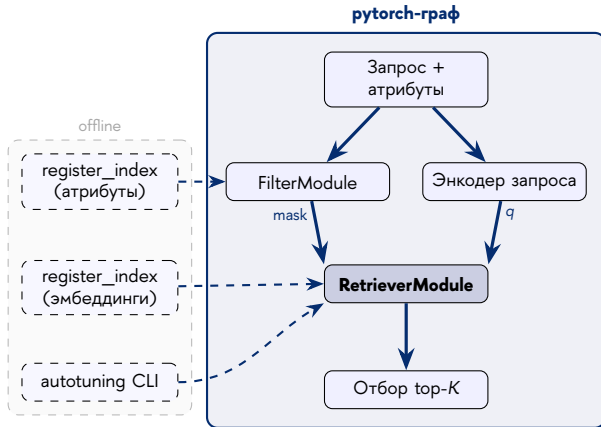
Фаза Filter — два варианта `FilterModule` (доступны любому алгоритму): **ExactAttributeFilter** — точная проверка по значениям, без ложных срабатываний; подходит при небольшом числе атрибутов и необходимости гарантированной полноты. **BloomFilter** — компактные 1024-битные сигнатуры и быстрый побитовый AND; оптимален для экономии памяти при большом числе атрибутов, допуская ложноположительные срабатывания (считаются нерелевантными).





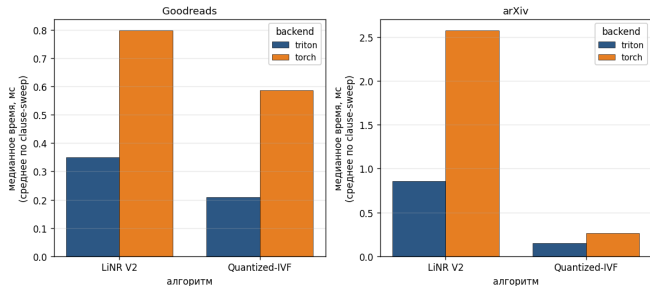
Архитектурные принципы

- Каждый алгоритм — стандартный `nn.Module`
- Единый интерфейс `FilterModule` для атрибутов
- Два бэкенда: оптимизированный Triton + эталонный PyTorch
- Ядра регистрируются через `@torch.library.triton_op`
- Отсутствие разрывов вычислительного графа (graph breaks)
- Предварительная (offline) автонастройка параметров ядер





Бэкенды Triton и PyTorch: ускорение по алгоритмам



Анализ результатов

- Медианное ускорение Triton-реализации по сравнению с эталонной на PyTorch
- PyTorch-бэкенд — эталон для верификации корректности

Бэкенды переключаются параметром `backend="triton"/"pytorch"` без перекомпиляции графа.



Метрики и эталоны

- $\text{Recall@K}(q) = |R_q \cap G_q| / \min(K, |G_q|)$
- С фильтрацией: эталон — точный KNN с фильтром
- Без фильтрации: эталон — реальные взаимодействия
- Время работы: медиана t_{med} по 4096 батчам
- Память: размер индекса + `max_memory_allocated`

Сетка экспериментов

- Размерность: $d \in \{64, 128, 256\}$
- $K \in \{100, 200, 400, 500, 1000\}$
- Батч: $B \in \{1, 8, 16\}$

Аппаратный стенд

- NVIDIA A100-SXM4-80GB
- Изолированный сервер, без сторонних задач во время замеров

Изоляция

- Каждая конфигурация — отдельный Python-процесс
- $N_{\text{warmup}} = 20$ итераций прогрева перед каждым замером
- Замеры времени через `triton.testing.do_bench`
- Компиляция: `torch.compile` в режиме `reduce-overhead, dynamic=True`



Наборы данных

Набор	Объектов	Запросов	Энкодер
Goodreads	797 K	313 K	gSASRec
arXiv	2,99 M	10 K	Nomic-Embed
Yambda-500M	3,06 M	46 K	gSASRec
Yambda-5B	5,37 M	459 K	gSASRec

Условия фильтрации (Goodreads/arXiv)

- **C0–C3** — одиночные атрибуты:
Goodreads: жанр, язык, формат, год;
arXiv: категория, лицензия, год, число версий
- **C4** — пара атрибутов (жанр+язык / категория+год)
- **C5** — объединение всех четырёх (наиболее строгое)

Селективность условий:

C0 на arXiv: $\approx 5\text{--}30\%$ каталога;

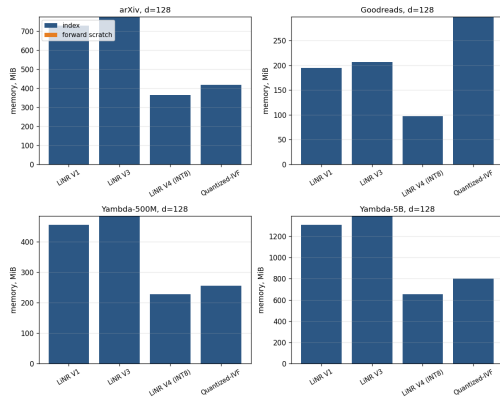
C5: $\lesssim 5\%$ — крайне строгий фильтр.



Результаты: поиск без фильтрации и потребление памяти

Набор (Recall@100)	LiNR V1	LiNR V4	LiNR V3	Quantized IVF
Goodreads	0,1479	0,1478	0,1438	0,1449
Yambda-500M	0,1486	0,1485	0,1426	0,1468
Yambda-5B	0,1779	0,1778	0,1590	0,1739

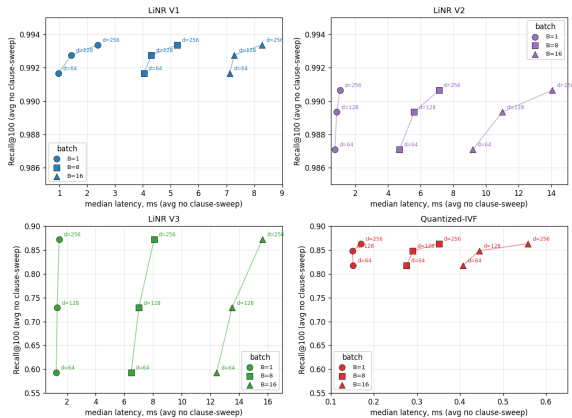
Recall@100, $d = 128$, $B = 1$, Triton, эталон — реальные взаимодействия пользователей.



Объем памяти индекса ($d = 128$).



Результаты: Парето-фронт качества, скорости и памяти (arXiv)



arXiv, $d = 128$, $K = 100$, $B = 1$.

Полнота, ускорение и память; $d = 128$, $K = 100$, $B = 1$.

arXiv

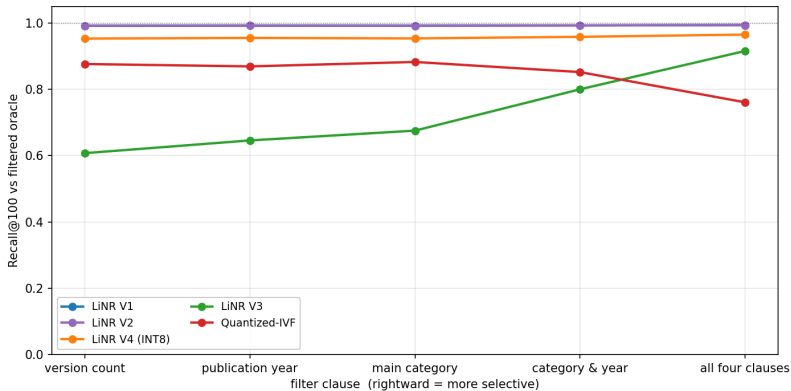
Усл.	Алгоритм	Recall	Уск.	МиБ
C0	LiNR V1	0,99	1,00×	730
	LiNR V2	0,99	1,68×	730
	LiNR V4	0,95	0,70×	365
	LiNR V3	0,68	1,08×	775
	QuantizedIVF	0,88	9,40×	419
C5	LiNR V2	0,99	1,76×	730
	LiNR V3	0,92	1,10×	775
	QuantizedIVF	0,76	9,47×	427

Goodreads (C0)

Алгоритм	Recall	Уск.	МиБ
LiNR V1	1,00	1,00×	195
LiNR V2	0,99	1,33×	195
LiNR V4	0,96	0,74×	97
LiNR V3	0,75	1,18×	207
QuantizedIVF	0,91	2,19×	298



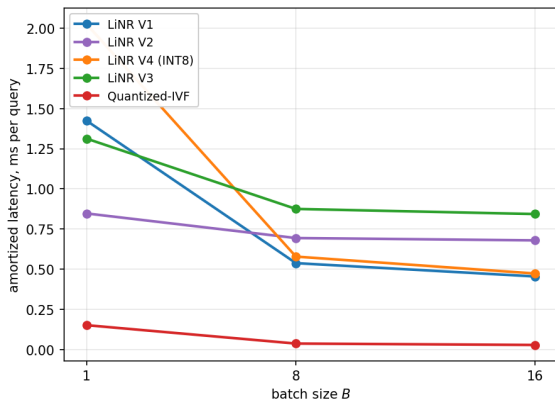
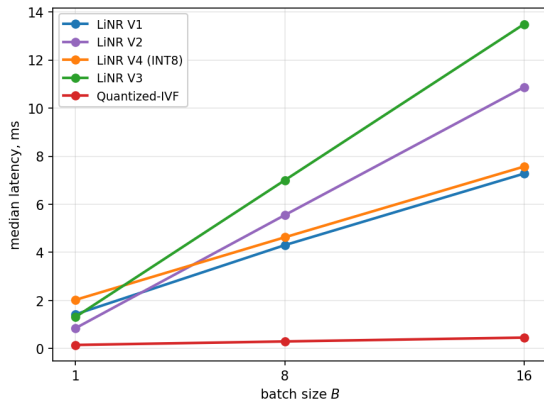
Устойчивость алгоритмов к селективности фильтра



Recall@100 в зависимости от селективности условия на arXiv ($d = 128, K = 100, B = 1$).



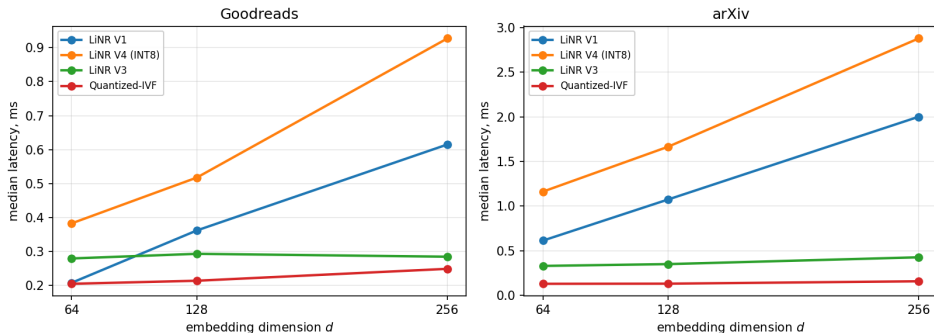
Масштабируемость алгоритмов



Амортизированное время (мс/запрос) от размера батча B (arXiv, $d = 128$, $K = 100$).



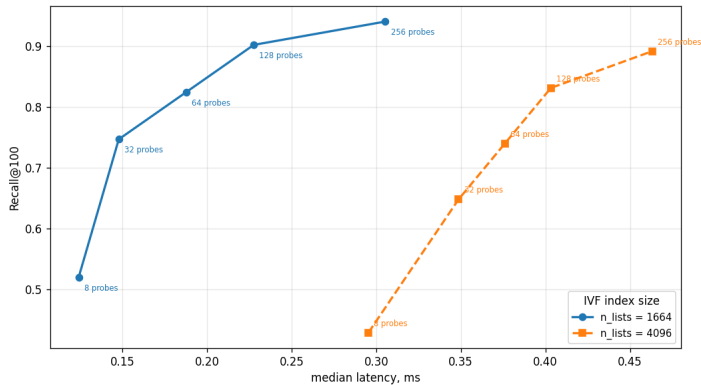
Зависимость скорости от размерности эмбедингов



Зависимость скорости алгоритмов от размерности векторов d .



Чувствительность к гиперпараметрам алгоритма QuantizedIVF



Парето-кривые QuantizedIVF при разном числе кластеров n_{lists} (arXiv, **C5**).



Результаты работы

1. **QuantizedIVF** — ускорение до $\approx 9\times$ при полноте 76–94%. Компактный индекс и гибкость делают его оптимальным выбором по умолчанию для крупных каталогов.
2. **LiNR V2** — оптимальный выбор при строгих фильтрах: полнота близка к идеальной, решает проблему нехватки кандидатов.
3. **LiNR V4** — размер индекса в два раза меньше при $\text{Recall} \geq 95\%$; при размере батча $B = 16$ время работы сравнивается с V1.
4. Фреймворк `torchretrieve` и все эксперименты опубликованы в открытом доступе — решена проблема воспроизводимости промышленных методов.

Направления дальнейшей работы

- Шардирование каталога между несколькими GPU (multi-GPU sharding)
- Потокное обновление индекса в реальном времени

Спасибо за внимание!